



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS

SEMESTRE 2027-1

PROFESORA: Jessica Santizo Galicia

AYUDANTE: Brenda Lizeth Ramirez Arista

Tabla de complejidades y Reflexión

Integrantes:

Galindo Ortis Marcos Santiago
Maldonado Solis Juan Alexis
Martinez Granados Alejandro
Urzúa Olais Aarón Felipe

Tabla de complejidades

Nombre del Problema	Estructura Principal utilizada	Complejidad temporal	Complejidad espacial
#1 Two Sum	Diccionario (Hash map)	O (n)	O(n)
# 56 Merge Intervals	Listas y algoritmo de ordenamiento	O (n log n)	O (n)
# 238 Product of Array Except Self	Listas (Variables acumuladoras)	O (n)	O (1) auxiliar
#65 Valid Number	Expresiones regulares (re.fullmath)	O(n)	O(1)
#592 Fraction Addition and Subtraction	Expresiones regulares (re.findall, re.split)	O(n)	O(n)
#929 Unique Email Addresses	Expresiones Regulares y Conjuntos (set)	O(n*m)	O(n*m)
#20 Valid Parentheses	Pila (list) y Diccionario	O(n)	O(n)
#150 Evaluate Reverse Polish Notation	Pila (list)	O(n)	O(n)
#739 Daily Temperatures	Pila Monótona (list)	O(n)	O(n)
#232 Implement Queue using Stacks	Dos pilas (list)	O(1) amortizado	O(n)
#622 Desing Circular Queue	Arreglo Fijo (list) y Aritmética Modular	O(1)	O(k)
#239 Sliding Window Maximum	Cola Monótona (collections.deque)	O(n)	O(k)

Reflexión del Equipo

Para comenzar analizamos a fondo el comportamiento de las estructuras de datos lineales en Python y su impacto directo en la eficiencia de nuestros algoritmos,, la primera gran diferencia de rendimiento que abordamos radica en por qué list.pop(0) posee una complejidad temporal de O(n) mientras que deque.popleft() opera en O(1). Esta disparidad se debe de forma fundamental a la manera en que cada estructura organiza los elementos dentro de la memoria RAM. La lista convencional de Python está implementada internamente sobre un arreglo dinámico contiguo de memoria, cuando se ejecuta el método list.pop(0), se elimina el primer elemento del arreglo, lo que genera un espacio vacío al inicio.

Para preservar la continuidad física de la memoria y asegurar que los índices sigan siendo estrictamente consecutivos, el intérprete de Python debe desplazar obligatoriamente cada uno de los elementos restantes una posición hacia la izquierda siendo que a este proceso de reubicación requiere n-1 operaciones de copia, volviendo al procedimiento lineal e ineficiente cuando se manejan volúmenes masivos de datos; En contraste, la estructura collections.deque (cola de doble extremo) se implementa internamente mediante una lista doblemente enlazada basada en bloques de memoria, en esta arquitectura los elementos no requieren ocupar posiciones contiguas.

Cada nodo mantiene referencias directas hacia el nodo anterior y el siguiente al llamar deque.popleft(), el sistema únicamente reasigna el puntero de la cabeza para que apunte al segundo nodo y libera la memoria del primero debido a que no existe necesidad alguna de desplazar los elementos adyacentes, la operación se completa en tiempo constante O(1). Durante la resolución del muestrario de ejercicios, el problema que demandó mayor esfuerzo conceptual para nuestro equipo fue el #239 Sliding Window Maximum.

Ya que inicialmente la aproximación intuitiva sugería iterar de forma directa sobre cada ventana flotante de tamaño K para extraer el máximo mediante la función max(), pero dicha estrategia derivó en una complejidad temporal ineficiente de O(n*k) que provocaba un fallo por tiempo límite de ejecución en la plataforma el desafío principal consistió en diseñar una estrategia que permitiera mantener el valor máximo de la ventana activo en tiempo constante mientras esta se desplazaba siendo que la solución definitiva se alcanzó tras comprender el concepto de cola monótona apoyada en un deque, la cual nos permitió almacenar los índices de los elementos candidatos en orden estrictamente decreciente.

De acuerdo a lo anterior el proceso fue eliminando por la derecha los valores menores al entrante y por la izquierda los que salían del rango de la ventana; Finalmente, al evaluar la validación de cadenas de texto con el módulo re, identificamos que la diferencia central entre re.fullmatch y re.match reside en el alcance de la verificación. La función re.match comprueba únicamente que el patrón coincida desde el inicio de la cadena, ignorando si el texto posterior contiene caracteres que violan el formato.

Por otro lado, re.fullmatch evalúa la cadena en su totalidad, exigiendo que coincida desde el primer hasta el último carácter. Por ejemplo, al validar un código postal con la expresión sobre la cadena, re.match devolverá un objeto exitoso se acoplara, mientras que re.fullmatch retornará None, evitando falsos positivos en la validación.